

JAVA WEB APPLICATION TECHNOLOGIES

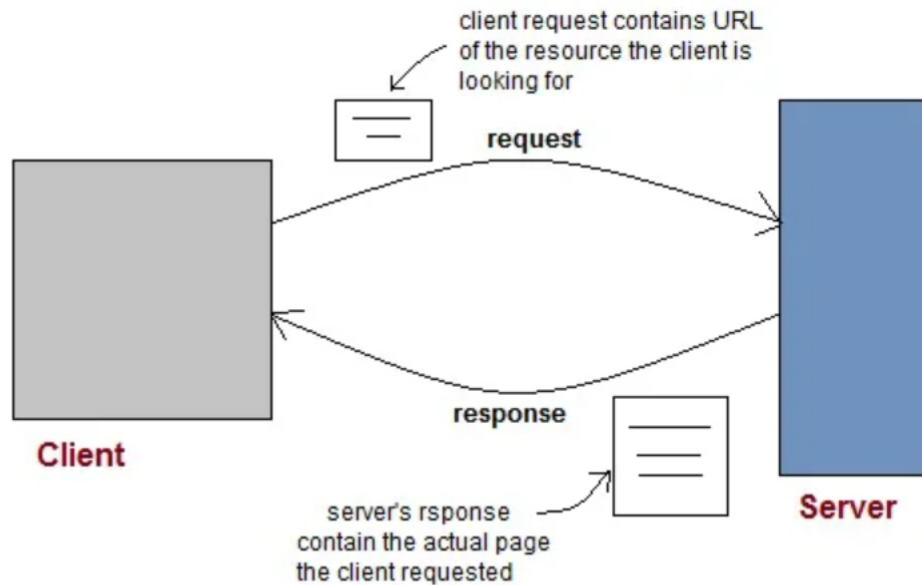


Introduction to Web

- Web consists of billions of clients and server connected through wires and wireless networks.
- The web clients make requests to web server.
- The web server receives the request, finds the resources and return the response to the client. When a server answers a request, it usually sends some type of content to the client.
- The client uses web browser to send request to the server. The server often sends response to the browser with a set of instructions written in HTML(HyperText Markup Language). All browsers know how to display HTML page to the client.



Introduction to Web



What is a web application

- A website is a collection of static files(webpages) such as HTML pages, images, graphics etc.
- A Web application is a web site with dynamic functionality on the server. Google, Facebook, Twitter are examples of web applications.



HTTP (Hypertext Transfer Protocol)

- HTTP is a protocol that clients and servers use on the web to communicate.
- It is stateless means each request is considered as the new request. In other words, server doesn't recognize the user by default.
- The client sends an HTTP request and the server answers with an HTML page to the client, using HTTP.



HTTP Methods

- HTTP request can be made using a variety of methods, but the ones you will use most often are Get and Post. The method name tells the server the kind of request that is being made, and how the rest of the message will be formatted.



HTTP Methods

Method Name	Description
OPTIONS	Request for communication options that are available on the request/response chain.
GET	Request to retrieve information from server using a given URI.
HEAD	Identical to GET except that it does not return a message-body, only the headers and status line.
POST	Request for server to accept the entity enclosed in the body of HTTP method.
DELETE	Request for the Server to delete the resource.
CONNECT	Reserved for use with a proxy that can switch to being a tunnel.
PUT	This is same as POST, but POST is used to create, PUT can be used to create as well as update. It replaces all current representations of the target resource with the uploaded content.



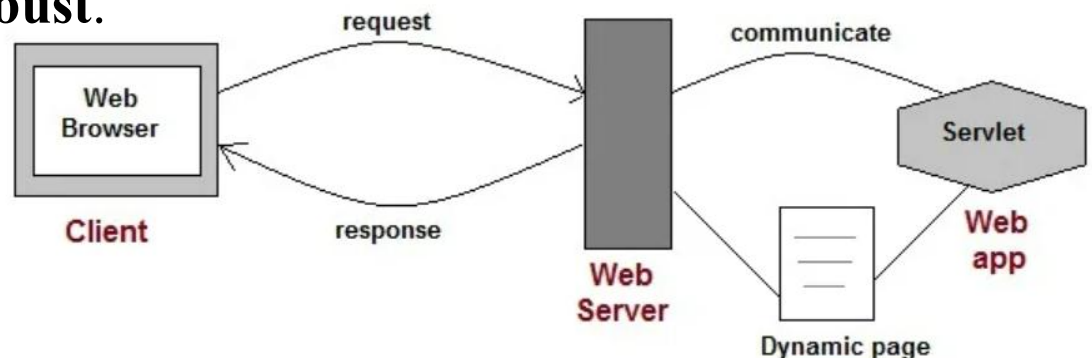
Difference between GET and POST requests

GET Request	POST Request
Data is sent in header to the server	Data is sent in the request body
Get request can send only limited amount of data	Large amount of data can be sent.
Get request is not secured because data is exposed in URL	Post request is secured because data is not exposed in URL.
Get request can be bookmarked and is more efficient.	Post request cannot be bookmarked.

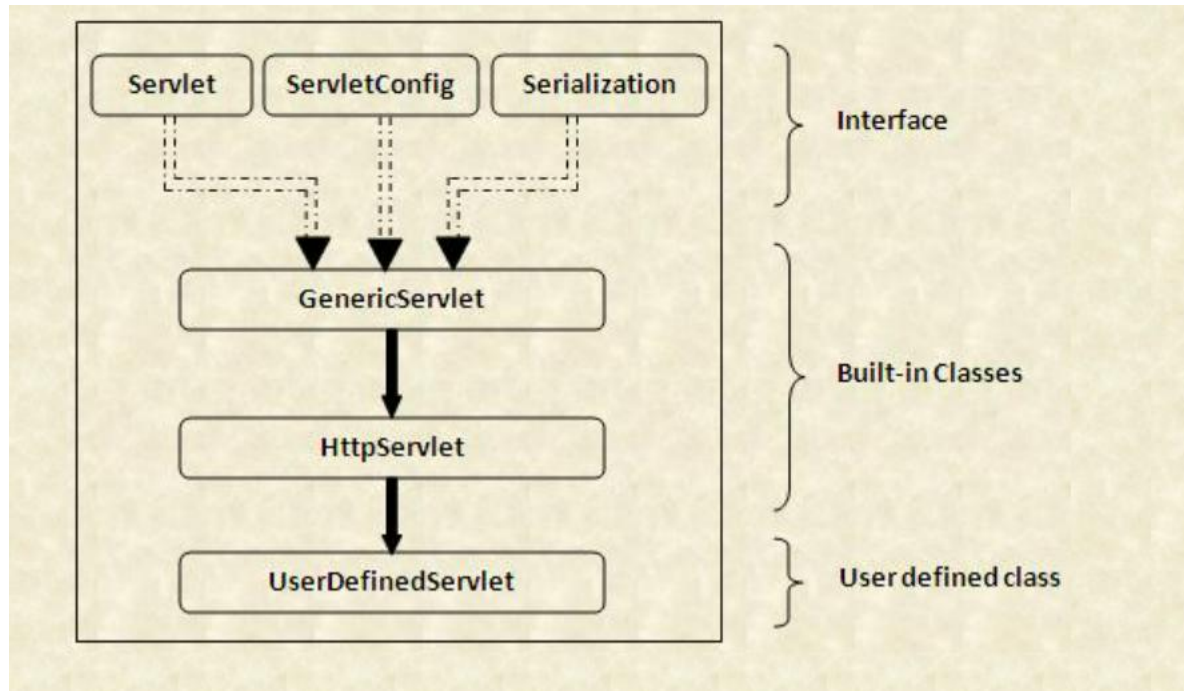


Introduction to Servlet

- **Servlet technology** is used to create **dynamic web applications**.
- It uses the **Java programming language** to process client requests on the server.
- A **web application** is a server-side application that generates **dynamic web content**.
- Dynamic web pages can change their content at runtime, such as:
 - displaying the **current date and time**
 - performing **calculations**
 - showing **user-specific information**
- Since servlets are built using Java, servlet-based web applications are:
Secure, Scalable, and Robust.



Servlet hierarchy



A servlet is an object that extends either the `javax.servlet.GenericServlet` class or the `javax.servlet.http.HttpServlet` class. The `javax.servlet.GenericServlet` class defines methods for building generic, protocol-independent servlets. The `javax.servlet.http.HttpServlet` class extends this class to provide HTTP-specific methods.

Advantages of Servlet

- **Better Performance**

Servlets use **multithreading**, so each request is handled by a thread instead of a new process.

- **Platform Independent (Portable)**

Since servlets are written in **Java**, they :

- automatic garbage collection can run on any server that supports the JVM.

- **Robust**

Servlets are managed by the JVM, which provides

- exception handling
- memory management

- **Secure**

Java provides built-in security features such as:

- strong type checking
- no direct pointer manipulation
- support for authentication and authorization



Advantages of Servlet

- **Efficient**

Servlets are loaded once and remain in memory, making request handling fast.

- **Scalable**

Servlets can handle a large number of simultaneous users efficiently.

- **Server Independent**

Servlets can run on any servlet container such as Tomcat, Jetty, or GlassFish.

- **Cost Effective**

Many servlet containers are **open-source** and free to use.

- **Protocol Independent (GenericServlet)**

Servlets can be protocol independent when implemented using GenericServlet, allowing support for protocols other than HTTP.

(In practice, most servlets extend HttpServlet, which is HTTP-specific.)



Role of servlets

- **Handling HTTP requests:** Servlets are designed to handle HTTP requests and responses. When a client (usually a web browser) sends a request to a web server, the server can delegate the processing of that request to a servlet. The servlet then generates a response, which is sent back to the client.
- **Dynamic content generation:** Servlets allow developers to generate dynamic content for web applications. Unlike static HTML pages, servlets can execute Java code and interact with databases or other resources to create dynamic content that can be personalized for each user.
- **Business logic implementation:** Servlets are commonly used to implement the business logic of a web application. They can process data, perform calculations, and make decisions based on user input or other parameters. Servlets are part of the server-side processing, which helps in separating business logic from the presentation layer.



Role of servlets

- **Handling forms and user input:** Servlets are often used to handle form submissions and process user input. They can extract data from HTML forms, validate it, and take appropriate actions based on the input.
- **Session management:** Servlets play a crucial role in managing user sessions. They can create, retrieve, and manage session information, allowing web applications to maintain state across multiple requests from the same user.
- **Handling authentication and authorization:** Servlets can be used to implement authentication and authorization mechanisms. They can check user credentials, validate access permissions, and enforce security policies to protect sensitive information.
- **Integration with other Java technologies:** Servlets are often used in conjunction with other Java technologies such as JavaServer Pages (JSP), JavaBeans, and Enterprise JavaBeans (EJB). This integration allows developers to build scalable and maintainable web applications.



What is a web container

- A **Web Container** is a component of a web server that **manages Java Servlets**.
- It provides the **runtime environment** for servlets.
- The web container is responsible for:
 - loading and instantiating servlets
 - managing the **servlet lifecycle**
 - mapping URLs to servlets
 - handling **multithreading**
 - managing **security**
- The web container receives client requests and passes them to the appropriate servlet.
- Example: Apache Tomcat



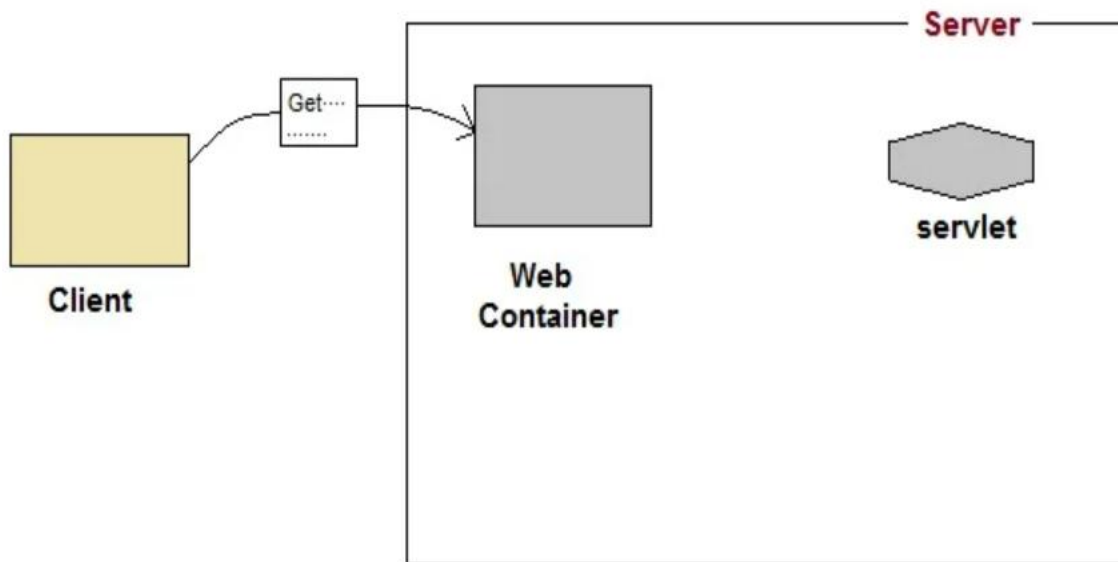
How a Servlet Application works

- When a request comes in for a servlet, the server hands the request to the Web Container.
- **Web Container** is responsible for instantiating the servlet or creating a new thread to handle the request. Its the job of Web Container to get the request and response to the servlet. The container creates multiple threads to process multiple requests to a single servlet.
- **Servlets don't have a main() method.** Web Container manages the life cycle of a Servlet instance.



How a Servlet Application works

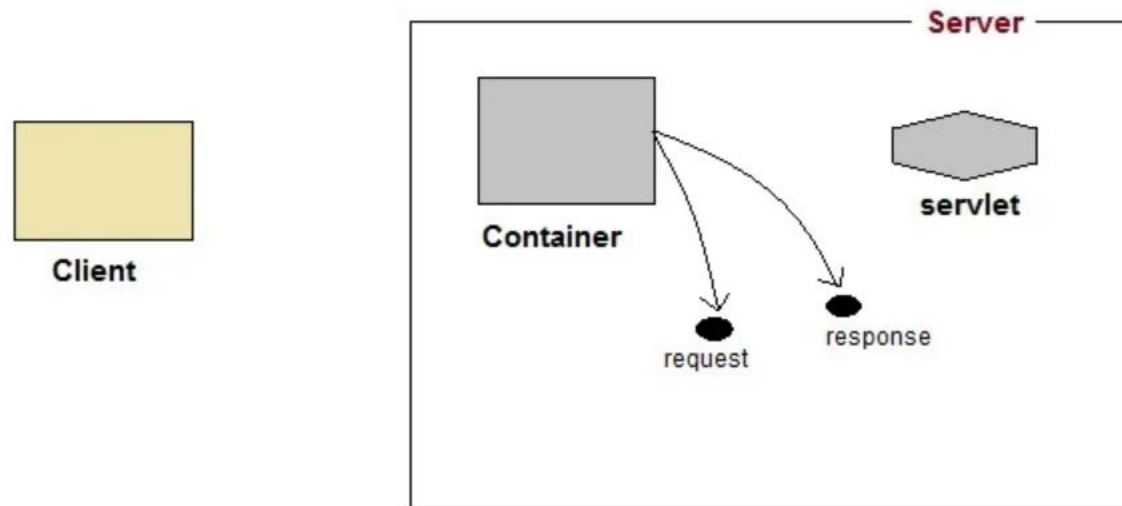
1. User sends request for a servlet by clicking a link that has URL to a servlet.



How a Servlet Application works

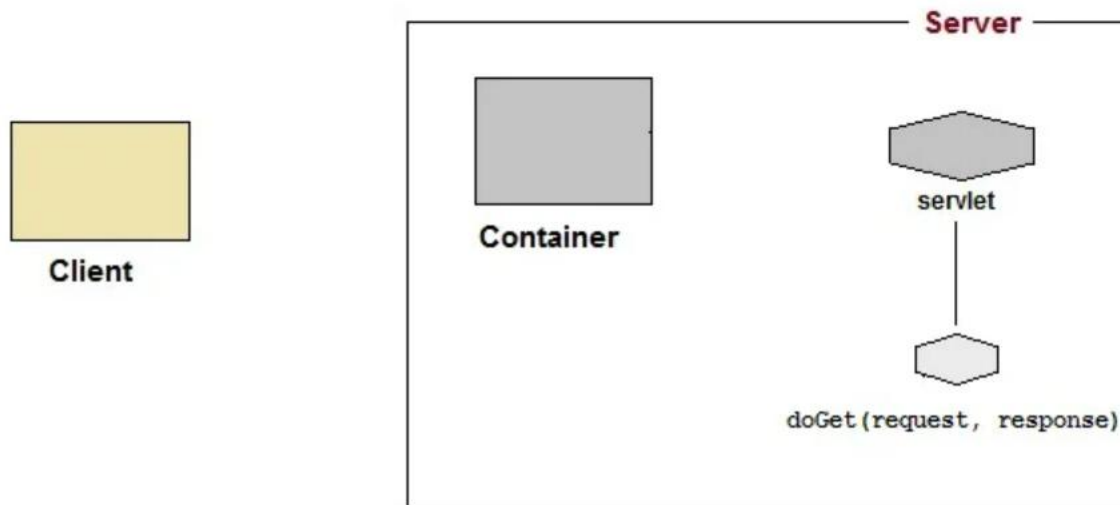
2. The container finds the servlet using **deployment descriptor** and creates two objects :

HttpServletRequest and HttpServletResponse



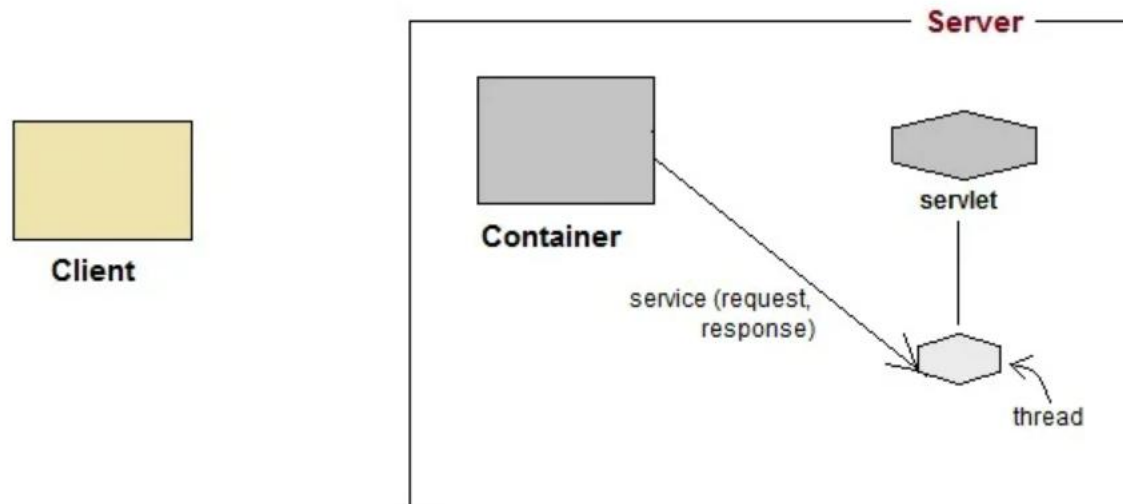
How a Servlet Application works

3. The **service()** method, then decides which servlet method, **doGet()** or **doPost()** to call, based on HTTP Request Method(Get, Post etc) sent by the client. Suppose the client sent an HTTP GET request, so the service() will call Servlet's doGet() method.



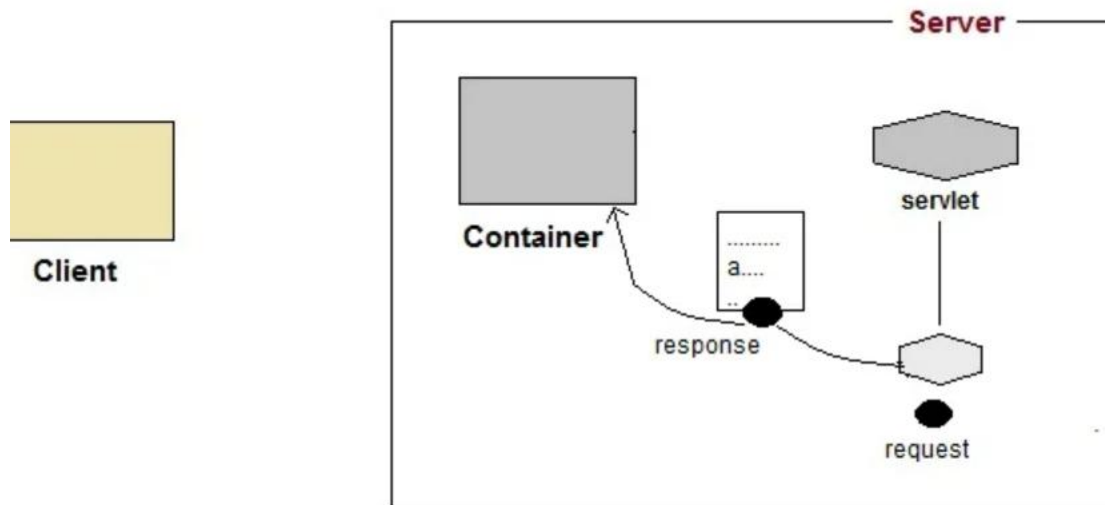
How a Servlet Application works

4. Then the container creates or allocates a thread for that request and calls the Servlet's `service()` method and passes the request, response objects as arguments.



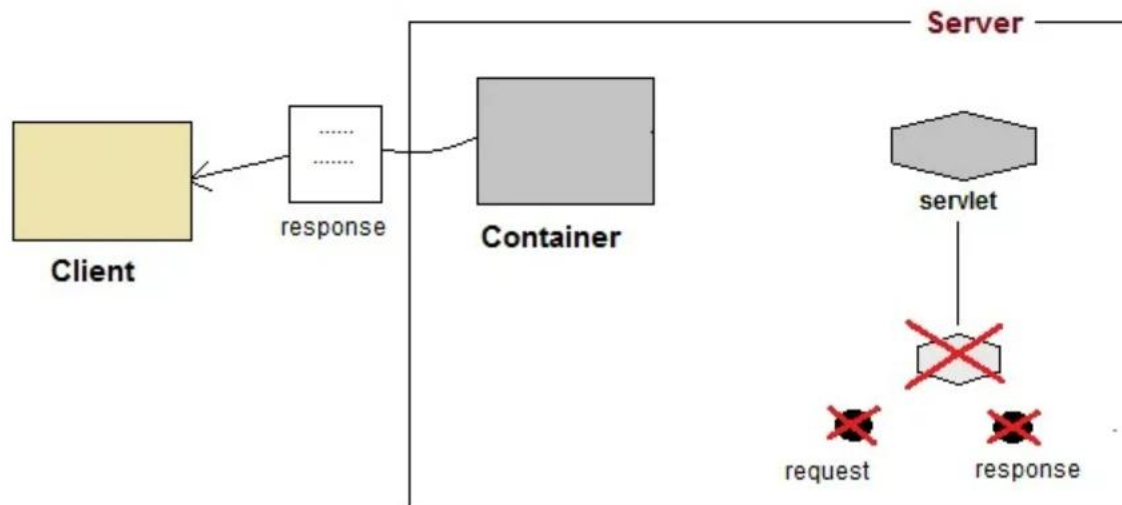
How a Servlet Application works

5. Then the Servlet uses response object to write the response back to the client.



How a Servlet Application works

6. After the **service()** method is completed the **thread** dies. And the request and response objects are ready for **garbage collection**



Deployment Descriptor

- In a java web application a file named web.xml is known as deployment descriptor. It is a xml file and `<web-app>` is the root element for it. When a request comes web server uses web.xml file to map the URL of the request to the specific code that handle the request.

Sample code of web.xml file:

```
<web-app>

    <servlet>
        <servlet-name>servletName</servlet-name>
        <servlet-class>servletClass</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>servletName</servlet-name>
        <url-pattern>*. *</url-pattern>
    </servlet-mapping>

</web-app>
```



Servlet life Cycle

Life cycle of a servlet is managed by web container:

- 1.Load Servlet Class.**
- 2.Create Servlet instance.**
- 3.Call init() method.**
- 4.Call service() method.**
- 5.Call destroy() method.**



Servlet life Cycle

- 1. Load Servlet Class:** Web container loads the servlet when the first request is received. This step is executed only once at the time of first request.
- 2. Create Servlet instance:** After loading the servlet class web container creates the servlet instance. Only one instance is created for a servlet and all concurrent requests are executed on the same servlet instance.
- 3. Call init() method:** After creating the servlet instance web container calls the servlet's init method. This method is used to initialize the servlet before processing first request. It is called only once by the web container



Servlet life Cycle

4. Call service() method: After initialization process web container calls service method. Service method is called for every request. For every request servlet creates a separate thread.

5. Call destroy() method: This method is called by web container before removing the servlet instance. Destroy method asks servlet to release all the resources associated with it. It is called only once by the web container when all threads of the servlet have exited or in a timeout case.



Servlet API

- The **Servlet API** is a set of interfaces and classes provided by Java to enable the development of web applications.
- It is part of the **Java EE (Jakarta EE)** platform and provides the foundation for building Java-based web applications.



Core Packages in Servlet API

- The Servlet API is primarily contained within two packages:
- **1. `javax.servlet`:** Provides classes and interfaces for handling generic requests and responses
- **2. `javax.servlet.http`:** Provides classes and interfaces specifically for handling HTTP protocol-based requests and responses.



Key Components of the Servlet API

- **1. Interfaces in javax.servlet**
- These are the general-purpose interfaces used by servlets:

Interface	Description
<code>Servlet</code>	The base interface for all servlets. Defines lifecycle methods (<code>init</code> , <code>service</code> , <code>destroy</code>).
<code>ServletConfig</code>	Provides configuration information to the servlet, such as initialization parameters.
<code>ServletContext</code>	Allows servlets to communicate with the servlet container (e.g., shared attributes, logging).
<code>RequestDispatcher</code>	Used for forwarding requests to other resources or including their
<code>Filter</code>	Used for filtering requests and responses, typically for preprocessing or postprocessing.
<code>FilterChain</code>	Allows multiple filters to be applied in sequence.
<code>AsyncContext</code>	Provides support for asynchronous processing in servlets.



Key Components of the Servlet API

- **2. Interfaces in javax.servlet.http**
- These are specific to HTTP-based web applications:

Interface	Description
<code>HttpServletRequest</code>	Represents the client's HTTP request. Provides methods for accessing request parameters, headers, etc.
<code>HttpServletResponse</code>	Represents the server's HTTP response. Provides methods to set status codes, headers, and response data.
<code>HttpSession</code>	Used to manage session data for a particular user across multiple requests.
<code>HttpSessionBindingListener</code>	Allows objects to receive notifications when added to or removed from a session.
<code>Cookie</code>	Represents an HTTP cookie, used for client-side state management.



ServletRequest

- The **ServletRequest** interface represents the client's request.
- It provides methods to:
- Retrieve parameters sent by the client (e.g., form data).
- Access input streams for reading request data.
- Retrieve information about the client and server.



Key Methods of ServletRequest

Method	Description
<code>getParameter(String name)</code>	Retrieves a parameter value sent in the request (e.g., from a form field).
<code>getParameterNames()</code>	Returns an <code>Enumeration</code> of all parameter names in the request.
<code>getParameterValues(String name)</code>	Returns an array of values for a parameter with multiple values.
<code>getInputStream()</code>	Gets the raw input stream for reading binary data.
<code>getReader()</code>	Returns a <code>BufferedReader</code> for reading text data.
<code>getContentType()</code>	Retrieves the MIME type of the request body (e.g., <code>text/html</code>).
<code>getContentLength()</code>	Returns the size of the request body in bytes.
<code>getRemoteAddr()</code>	Retrieves the IP address of the client making the request.
<code>getLocalAddr()</code>	Retrieves the server's IP address handling the request.



ServletResponse

- The **ServletResponse** interface represents the server's response.
- It provides methods to:
- Send data back to the client (HTML, JSON, etc.).
- Set response headers and status codes.
- Specify the content type (e.g., text/html, application/json).



Key Methods of ServletResponse

Method	Description
<code>setContentType(String type)</code>	Specifies the MIME type of the response (e.g., <code>text/html</code> , <code>application/json</code>).
<code>getWriter()</code>	Returns a <code>PrintWriter</code> to write character-based data (e.g., HTML).
<code>getOutputStream()</code>	Returns a <code>ServletOutputStream</code> to write binary data (e.g., images, PDFs).
<code>setBufferSize(int size)</code>	Sets the buffer size for the response data.
<code>setCharacterEncoding(String enc)</code>	Sets the character encoding of the response (e.g., <code>UTF-8</code>).



Servlet Configuration

- Servlet configuration involves providing custom settings or parameters to a servlet.
- This is done using:
- **ServletConfig Interface:** Used for servlet-specific configuration.
- **ServletContext Interface:** Used for application-wide configuration.



ServletConfig VS ServletContext in Java Servlet

- **ServletConfig** and **ServletContext**, both are objects created at the time of servlet initialization and used to provide some initial parameters or configuration information to the servlet. But, the difference lies in the fact that information shared by ServletConfig is for a specific servlet, while information shared by ServletContext is available for all servlets in the web application
- ServletConfig is an object containing some initial parameters or configuration information created by Servlet Container and passed to the servlet during initialization.
- ServletConfig is for a particular servlet, that means one should store servlet specific information in web.xml and retrieve them using this object.
- ServletContext is the object created by Servlet Container to share initial parameters or configuration information to the whole application.



Servlet example

```
1 package com.rca.exercises;
2
3 import javax.servlet.http.HttpServlet;
4
5 public class Calculations extends HttpServlet {
6     public void service(ServletRequest req, ServletResponse res) throws IOException {
7         int fistnum=Integer.parseInt(req.getParameter("num1"));
8         int secondnum=Integer.parseInt(req.getParameter("num2"));
9         int sum=fistnum+secondnum;
10        PrintWriter out=res.getWriter();
11        res.setContentType("text/html");
12        out.println("the answer is"+sum);
13        System.out.println("the sum is"+sum);
14    }
15 }
16 }
```

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://xml
  <display-name>Year Two WebAPP</display-name>
  <servlet>
    <servlet-name>calculations</servlet-name>
    <servlet-class>com.rca.exercises.Calculations</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>calculations</servlet-name>
    <url-pattern>/add</url-pattern>
  </servlet-mapping>
```

```
</web-app>
```



Servlet example

```
<body>  
<form action="add">  
  First number:<input type="text" name="num1"><br>  
  Second number:<input type="text" name="num2"><br>  
  <input type="submit">  
</form>  
</body>  
</html>
```



Servlet Communication

- **Servlet communication** refers to the process by which servlets share information or interact with one another within a web application. It is essential for implementing complex web application workflows where multiple servlets need to work together.
- **Methods of Servlet Communication**
 1. Request Dispatching
 2. Redirecting
 3. Session Management
 4. Application Context (ServletContext)
 5. Request Attributes



Servlet collaboration:

1.RequestDispatcher in Servlet

- Servlet collaboration refers to the process where multiple servlets communicate with each other to handle a request or share data. This is useful in situations where different components of an application need to work together to generate a response or perform complex tasks.

- There are two methods defined in the RequestDispatcher interface.

1.public void forward(ServletRequest request,ServletResponse response)throws ServletException,java.io.IOException:Forwards a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server.

2.public void include(ServletRequest request,ServletResponse response)throws ServletException,java.io.IOException:Includes the content of a resource (servlet, JSP page, or HTML file) in the response.



Servlet collaboration:

1.RequestDispatcher in Servlet

- The RequestDispatcher interface provides the facility of dispatching the request to another resource it may be html, servlet or jsp. This interface can also be used to include the content of another resource also. It is one of the way of servlet collaboration.

- There are two methods defined in the RequestDispatcher interface.

1.public void forward(ServletRequest request,ServletResponse response)throws ServletException,java.io.IOException:Forwards a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server.

2.public void include(ServletRequest request,ServletResponse response)throws ServletException,java.io.IOException:Includes the content of a resource (servlet, JSP page, or HTML file) in the response.



Example of RequestDispatcher

- We have a page which takes 2 numbers and calculate the sum/difference based on the choice of the user
- The request is sent to Calculations servlet.
- The Calculations servlet will dispatch the request to Sum or Subtraction servlets based on the selection of the user

First number:

Second number:

☐ Add

☐ Difference



Example of RequestDispatcher

```
package com.rca.exercises;

import javax.servlet.http.*;

public class Calculations extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {

        ServletConfig con=getServletConfig();
        String email=con.getInitParameter("email");
        System.out.println("the email is"+email);

        int fistnum=Integer.parseInt(req.getParameter("num1"));
        int secondnum=Integer.parseInt(req.getParameter("num2"));
        int calculationType=Integer.parseInt(req.getParameter("calculation"));
        System.out.println("type is"+calculationType);
        int sum=fistnum+secondnum;

        RequestDispatcher rd;
        req.setAttribute("fistnum", fistnum);
        req.setAttribute("secondnum", secondnum);
        if(calculationType==1) {
            rd=req.getRequestDispatcher("add");
            rd.forward(req, res);
        }else {
            rd=req.getRequestDispatcher("subtraction");
            rd.forward(req, res);
        }
    }
}
```



Example of RequestDispatcher

```
1 package com.rca.exercises;
2
3 import java.io.IOException;
4
5 public class Subtraction extends HttpServlet{
6     public void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
7         System.out.println("Subtraction is called");
8         int a=(int)req.getAttribute("firstnum");
9         int b=(int)req.getAttribute("secondnum");
10        PrintWriter out=res.getWriter();
11        res.setContentType("text/html");
12        int sum=a-b;
13        out.println("the Difference is"+sum);
14        System.out.println("the Difference is"+sum);
15    }
16 }
```

First number:

Second number:

☐ Add

☐ Difference

```
package com.rca.exercises;

import java.io.IOException;

public class Sum extends HttpServlet{
    public void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        System.out.println("add is called");
        int a=(int)req.getAttribute("firstnum");
        int b=(int)req.getAttribute("secondnum");
        PrintWriter out=res.getWriter();
        res.setContentType("text/html");
        int sum=a+b;
        out.println("the Sum is"+sum);
        System.out.println("the sum is"+sum);
    }
}
```

/23/2026

2.SendRedirect in servlet

- The **sendRedirect()** method of **HttpServletResponse** interface can be used to redirect response to another resource, it may be servlet, jsp or html file.
- It accepts relative as well as absolute URL.
- It works at client side because it uses the url bar of the browser to make another request. So, it can work inside and outside the server.



sendRedirect() and Request Dispatcher

- The main difference between a **redirection** and a **request dispatching** is that, redirection makes the client(browser) create a new request to get to the resource, the user can see the new URL while request dispatch get the resource in same request and URL does not changes.
- Also, another very important difference is that, sendRedirect() works on **response** object while request dispatch work on **request** object.



Exercise

- Create an Html file which receives the request from use, then redirect the request to google server. (use servlet)



Attribute in servlet

An **attribute in servlet** is an object that can be set, get or removed from one of the following scopes:

1.request scope

2.session scope

3.application scope

- The servlet programmer can pass informations from one servlet to another using attributes. It is just like passing object from one class to another so that we can reuse the same object again and again.



Attribute specific methods of ServletRequest, HttpSession and ServletContext interface

There are following 4 attribute specific methods. They are as follows:

- 1.**public void setAttribute(String name, Object object):**sets the given object in the application scope.
- 2.**public Object getAttribute(String name):**Returns the attribute for the specified name.
- 3.**public Enumeration getInitParameterNames():**Returns the names of the context's initialization parameters as an Enumeration of String objects.
- 4.**public void removeAttribute(String name):**Removes the attribute with the given name from the servlet context.



Session Management in Servlet

- We all know that **HTTP** is a stateless protocol. All requests and responses are independent. But sometimes you need to keep track of client's activity across multiple requests. For eg. When a User logs into your website, not matter on which web page he visits after logging in, his credentials will be with the server, until he logs out. So this is managed by creating a session.
- **Session Management** is a mechanism used by the **Web container** to store session information for a particular user. There are four different techniques used by Servlet application for session management:

1. **Cookies**

2. **Hidden form field**

3. **URL Rewriting**

4. **HttpSession**



Servlet: What is HttpSession?

- **HttpSession** object is used to store entire session with a specific client. We can store, retrieve and remove attribute from HttpSession object. Any servlet can have access to **HttpSession** object throughout the getSession() method of the HttpServletRequest object.
- **How HttpSession works**
 1. On client's first request, the **Web Container** generates a unique session ID and gives it back to the client with response. This is a temporary session created by web container.
 2. The client sends back the session ID with each request. Making it easier for the web container to identify where the request is coming from.
 3. The **Web Container** uses this ID, finds the matching session with the ID and associates the session with the request.



Servlet: HttpSession Interface

- 1. Get or Create a Session

```
HttpSession session = request.getSession();
```

- Returns an existing session if it exists
- Creates a new session if it does not exist
(Equivalent to getSession(true))

- 2. Get Existing Session Only

```
HttpSession session = request.getSession(false);
```

- Returns the session if it exists
- Returns null if no session exists
- Does NOT create a new session
- Used to check if user is already logged in.



Servlet: HttpSession Interface

- 3. Destroy a Session
- `session.invalidate();`
- Ends the session
- Removes all session data
- Logs the user out



HttpSession example

- Let's consider our example of adding 2 numbers

```
public void doGet(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {  
  
    ServletConfig con=getServletConfig();  
    ServletContext cxt=getServletContext();  
    String email=con.getInitParameter("email");  
    String website=cxt.getInitParameter("website");  
    System.out.println("the email is"+email+"website:"+website);  
  
    int fistnum=Integer.parseInt(req.getParameter("num1"));  
    int secondnum=Integer.parseInt(req.getParameter("num2"));  
    int calculationType=Integer.parseInt(req.getParameter("calculationType"));  
    System.out.println("type is"+calculationType);  
    int sum=fistnum+secondnum;  
  
    RequestDispatcher rd;  
    req.setAttribute("fistnum", fistnum);  
    req.setAttribute("secondnum", secondnum);  
    if(calculationType==1) {  
        HttpSession session = req.getSession();  
        session.setAttribute("fistnum", fistnum);  
        session.setAttribute("secondnum", secondnum);  
        res.sendRedirect("add");  
    }  
}
```



HttpSession example

- Sum servlet

```
public class Sum extends HttpServlet{
    public void doGet(HttpServletRequest req, HttpServletResponse res)
    {
        System.out.println("add is called");
        /*
         * int a=(int)req.getAttribute("fistnum"); int
         * b=(int)req.getAttribute("secondnum");
         */
        HttpSession session = req.getSession();
        int firstNum=(int)session.getAttribute("fistnum");
        int secondNum=(int)session.getAttribute("secondnum");
        PrintWriter out=res.getWriter();
        res.setContentType("text/html");
        int sum=firstNum+secondNum;
        out.println("the Sum is"+sum);
        System.out.println("the sum is"+sum);
    }
}
```



HttpSession example

- Let's consider our example of adding 2 numbers

```
1 <servlet-name>calculations</servlet-name>
2   <servlet-class>com.rca.exercises.Calculations</servlet-class>
3   <init-param>
4     <param-name>email</param-name>
5     <param-value>larimahoro@gmail.com</param-value>
6   </init-param>
7 </servlet>
8 <servlet-mapping>
9   <servlet-name>calculations</servlet-name>
10  <url-pattern>/calculus</url-pattern>
11 </servlet-mapping>
12 <servlet>
13   <servlet-name>sum</servlet-name>
14   <servlet-class>com.rca.exercises.Sum</servlet-class>
15 </servlet>
16 <servlet-mapping>
17   <servlet-name>sum</servlet-name>
18   <url-pattern>/add</url-pattern>
19 </servlet-mapping>
```



Using Cookies for Session Management in Servlet

- **Cookies** are small pieces of information that are sent in response from the web server to the client. **Cookies** are the simplest technique used for storing client state.
- **Cookies** are stored on client's computer. They have a lifespan and are destroyed by the client browser at the end of that lifespan.
- Using Cookies for storing client state has one shortcoming though, if the client has turned off Cookie saving settings in his browser then, client state can never be saved because the browser will not allow the application to store cookies.



Servlet: Cookies API

- Cookies are created using **Cookie** class present in Servlet API. Cookies are added to **response** object using the **addCookie()** method. This method sends cookie information over the HTTP response stream. **getCookies()** method is used to access the cookies that are added to response object

Creating a new Cookie

```
Cookie ck = new Cookie("username", name);
```

creating a new cookie object

Sending the cookie to the client

```
response.addCookie(ck);
```

adding cookie to response object

Getting cookies from client request

```
Cookie[] cks = request.getCookies();
```

getting the cookie for request object



1/23/2026

Using cookies

- Let's use the previous example for adding two numbers, and send the request to Sum class

```
if(calculationType==1) {  
    //HttpSession session = req.getSession();  
    Cookie fistNUM= new Cookie("fistnum", fistnum+"");  
    Cookie secondNUM= new Cookie("secondnum", secondnum+"");  
    res.addCookie(fistNUM);  
    res.addCookie(secondNUM);
```

Calculati
on class

```
res.sendRedirect("add");
```

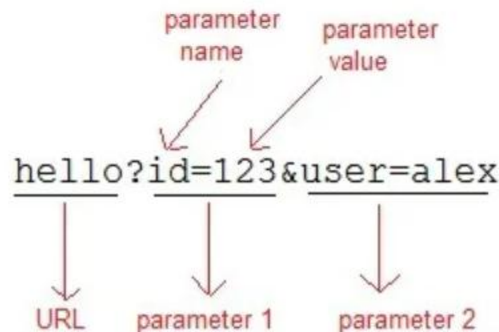
```
Cookie cookies[]=req.getCookies();  
int firstNum=Integer.parseInt(cookies[0].getValue());  
int secondNum=Integer.parseInt(cookies[1].getValue());  
PrintWriter out=res.getWriter();  
res.setContentType("text/html");  
int sum=firstNum+secondNum;  
out.println("the Sum is"+sum);  
System.out.println("the sum is"+sum);
```

Sum class



Using URL Rewriting for Session Management in Servlet

- If the client has disabled cookies in the browser then session management using cookie wont work. In that case **URL Rewriting** can be used as a backup. URL rewriting will always work.
- In URL rewriting, a token(parameter) is added at the end of the URL. The token consist of name/value pair seperated by an **equal(=)** sign.
- The **getParameter()** method is used to get the parameter value at the server side.



Using Hidden Form Field for Session Management in Servlet

- Hidden form field can also be used to store session information for a particular client. In case of hidden form field a hidden field is used to store client state. In this case user information is stored in hidden field value and retrieved from another servlet.

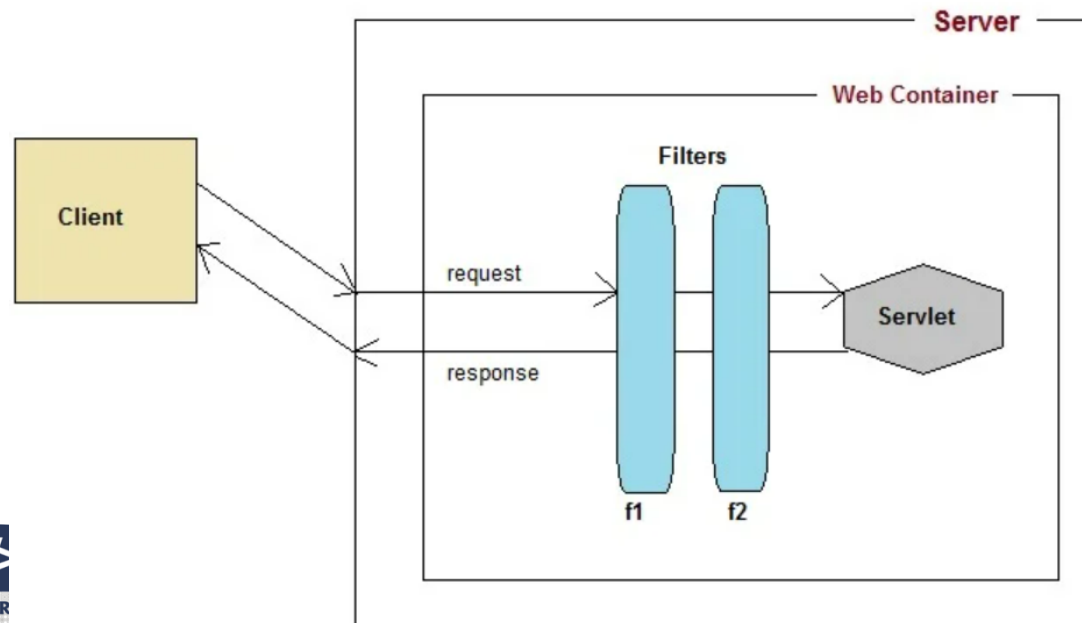
Disadvantage of Using Hidden Form Field for Session Management

- Extra form submission is required on every page. This is a big overhead.



Servlet: Filters

- **Filters** are components that you can use and configure to perform some filtering tasks. Filter is used for pre-processing of requests and post-processing of responses. You can have any number of filters for pre-processing of a request and post-processing of a response. Filters are configured in the deployment descriptor of a web application.



How Filters Works?

- When a request reaches the **Web Container**, it checks if any filter has URL patterns that matches the requested URL.
- The **Web Container** locates the first filter with a matching URL pattern and filter's code is executed.
- If another filter has a matching URL pattern, its code is then executed. This continues until there are no filters with matching URL patterns left.
- If no error occurs, the request passes to the target servlet. Hence we know, that the request will be passed to the target servlet only when all the related Filters are successfully executed.
- The servlet returns the response back to its caller. The last filter that was applied to the request is the first filter applied to the response.
- At last the response will be passed to the **Web Container** which passes it to the client.



Servlet: Filter API

- **Filter API** is part of **Servlet API**. Filter interface is found in the **javax.servlet** package.
- For creating a filter, we must implement Filter interface. Filter interface gives the following life cycle methods for a filter:
 1. **void init**(FilterConfig filterConfig): invoked by the web container to indicate to a filter that it is being placed into service.
 2. **void doFilter**(ServletRequest request, ServletResponse response, FilterChain chain): invoked by the container each time a request/response pair is passed through the chain due to a client request for a resource at the end of the chain.
 3. **void destroy**(): invoked by the web container to indicate to a filter that it is being taken out of service.



Servlet: What is FilterChain Interface?

- **FilterChain** object is used to invoke the next filter in the chain, or if the calling filter is the last filter in the chain then the resource at the end of the chain invoked. The resources at the end of Filter chain can either be a target Servlet(in case of request flow) or the Client(in case of response flow) as described in the diagram above.

Declaring a Servlet Filter inside Deployment Descriptor

```
<web-app ...>
<filter>
  <filter-name>MyFilter</filter-name>
  <filter-class>MyFilter</filter-class>
</filter>
<filter-mapping>
  <filter-name>MyFilter</filter-name>
  <url-pattern>..</url-pattern> or <servlet-name>.</servlet-name>
</filter-mapping>
</web-app>
```

<filter-name> tag is used to give a internal name to your filter

<filter-class> declares the filter that you have created

Either the <url-pattern> or the <servlet-name> element is mandatory which web app resource will use this filter



1/23/2026

Filters example

```
@Override
public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
    throws IOException, ServletException {
    // TODO Auto-generated method stub
    System.out.println("in filter");
    int firstNum=0;
    HttpServletRequest req=(HttpServletRequest)request;
    Cookie cookies[]=req.getCookies();
    for (Cookie c:cookies) {
        if(c.getName().equals("fistnum")) {
            firstNum=Integer.parseInt(c.getValue());
        }
    }

    System.out.println("in filter"+firstNum);
    if(firstNum<1) {
        PrintWriter out=response.getWriter();
        response.setContentType("text/html");
        out.print("use positive numbers");
    }else
        chain.doFilter(request, response);
}
```



Filters example

```
</context-param>  
<filter>  
  <filter-name>addfilter</filter-name>  
  <filter-class>com.rca.filters.AddFilter</filter-class>  
</filter>  
<filter-mapping>  
  <filter-name>addfilter</filter-name>  
  <url-pattern>/add</url-pattern>  
</filter-mapping>
```

